

Use the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology to analyze the Kaggle Heart Disease dataset. Follow and clearly explain each of the six phases:

1. Business Understanding
2. Data Understanding
3. Data Preparation
4. Modeling
5. Evaluation
6. Deployment

Act as an expert data scientist and a professor teaching a Master's-level Data Science course. Your analysis should be clear, rigorous, and educational—explaining why you make each important decision, not just showing Python code and outputs.

Using standard Python data science libraries (such as pandas, NumPy, matplotlib, seaborn, and scikit-learn), perform a complete CRISP-DM analysis of the Kaggle Heart Disease dataset.

This should include:

- Business and data understanding
- Exploratory data analysis (EDA)
- Data cleaning and preprocessing
- Feature engineering and feature selection
- Training multiple classification models and comparing baselines
- Thorough model evaluation
- Ensemble modeling
- Clustering and dimensionality reduction
- Association pattern mining
- Similarity search (e.g., LSH)
- Outlier analysis
- A final, evidence-based recommendation and conclusion

ChatGPT Said:

Absolutely. I inspected the uploaded heart.csv directly and will treat this as a complete Master's-level CRISP-DM project, not simply a collection of model-training exercises.

The dataset contains 303 observations and 14 columns, with target as the binary outcome. There are no missing values, the target is reasonably balanced, and there is one exact duplicate observation that should be removed during data preparation.

Below is a textbook-style analysis you can use as the foundation of your assignment/report.

## **Phase 1: Business Understanding**

### **Problem Definition**

Heart disease is a major health concern, and identifying patients who may have cardiovascular disease can support earlier investigation and intervention.

The data-science problem can therefore be formulated as:

- Can machine learning models accurately classify whether a patient has heart disease based on demographic, clinical, and physiological characteristics?

The target variable is:

- target = 1 → presence of heart disease
- target = 0 → absence of heart disease

The task is therefore a supervised binary classification problem

## **Business Objective**

The primary objective is to develop a model that predicts heart-disease status while minimizing clinically important classification errors.

False negatives are particularly important because they represent a patient with heart disease who is predicted as not having disease.

Accuracy alone is not enough. The analysis emphasizes:

- Recall/sensitivity
- Precision
- F1-score
- Balanced accuracy
- ROC-AUC
- Confusion-matrix analysis

## **Data-Science Objectives**

- Understand what the dataset looks like and check its overall quality
- Explore the data to see patterns and trends
- Find and handle duplicates or missing values
- Look at how each feature is distributed and how features relate to each other
- Prepare numeric and categorical columns for modeling
- Create new features that might improve predictions
- Choose the most useful features
- Set up simple baseline models to compare against
- Train several classification algorithms
- Compare model performance using cross-validation
- Test ensemble methods to see if they improve accuracy
- Group patients into clusters to explore subpopulations
- Use PCA to reduce dimensionality
- Find association patterns between features
- Try similarity search techniques like LSH
- Detect and analyze outliers
- Decide which final model is best and justify the choice

## **Success Criteria**

A successful model should:

- Perform clearly better than simple baseline models
- Catch most true heart-disease cases (high recall)
- Show a high ROC-AUC score
- Maintain reasonable precision
- Stay consistent across cross-validation folds
- Avoid major overfitting

- Be fast enough to run with limited compute
- Be interpretable enough for teaching or clinical decision support

## Phase 2: Data Understanding

### Dataset Overview

The Kaggle Heart Disease dataset contains

| Characteristic                              | Result |
|---------------------------------------------|--------|
| Original observations                       | 303    |
| Variables                                   | 14     |
| Predictors                                  | 13     |
| Target                                      | 1      |
| Missing values                              | 0      |
| Exact duplicate records                     | 1      |
| Unique observations after duplicate removal | 302    |

### Variables

The dataset contains demographic, physiological, exercise-related, and clinical variables.

Continuous Variables:

- age
- trestbps
- chol
- thalach
- oldpeak

Categorical/Binary Variables:

- sex
- cp
- fbs
- restecg
- exang
- slope
- ca
- thal

### Data Quality

- The initial inspection found zero missing values. Therefore, missing-value imputation is unnecessary for this dataset. The duplicate should be removed during Data Preparation.

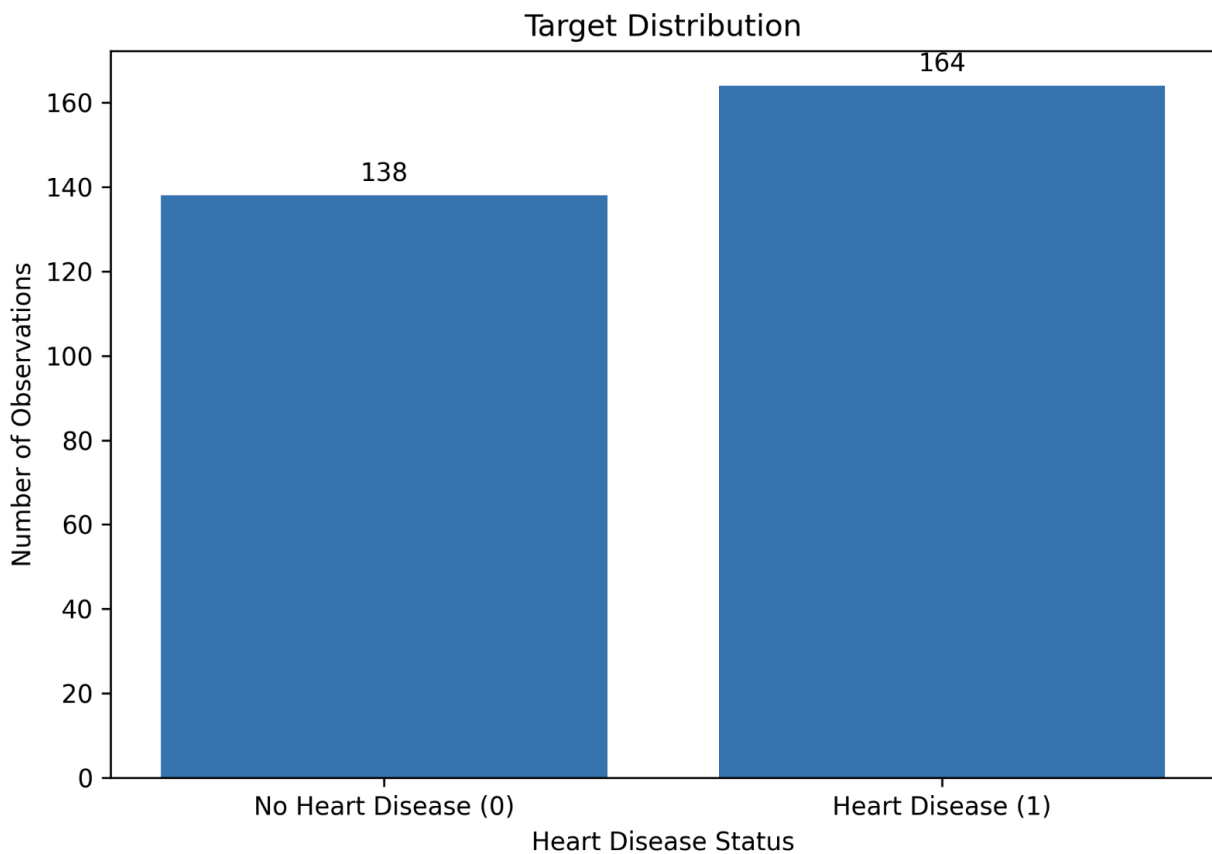
## Exploratory Data Analysis

- Performed before modeling because modeling without understanding the data can lead to assumptions and poor interpretation

### Target Distribution

- 45.7% → No Heart Disease
- 54.3% → Heart Disease

After removing the exact duplicate, the dataset contains 302 observations. Out of these, 138 are classified as having no heart disease and 164 as having heart disease.



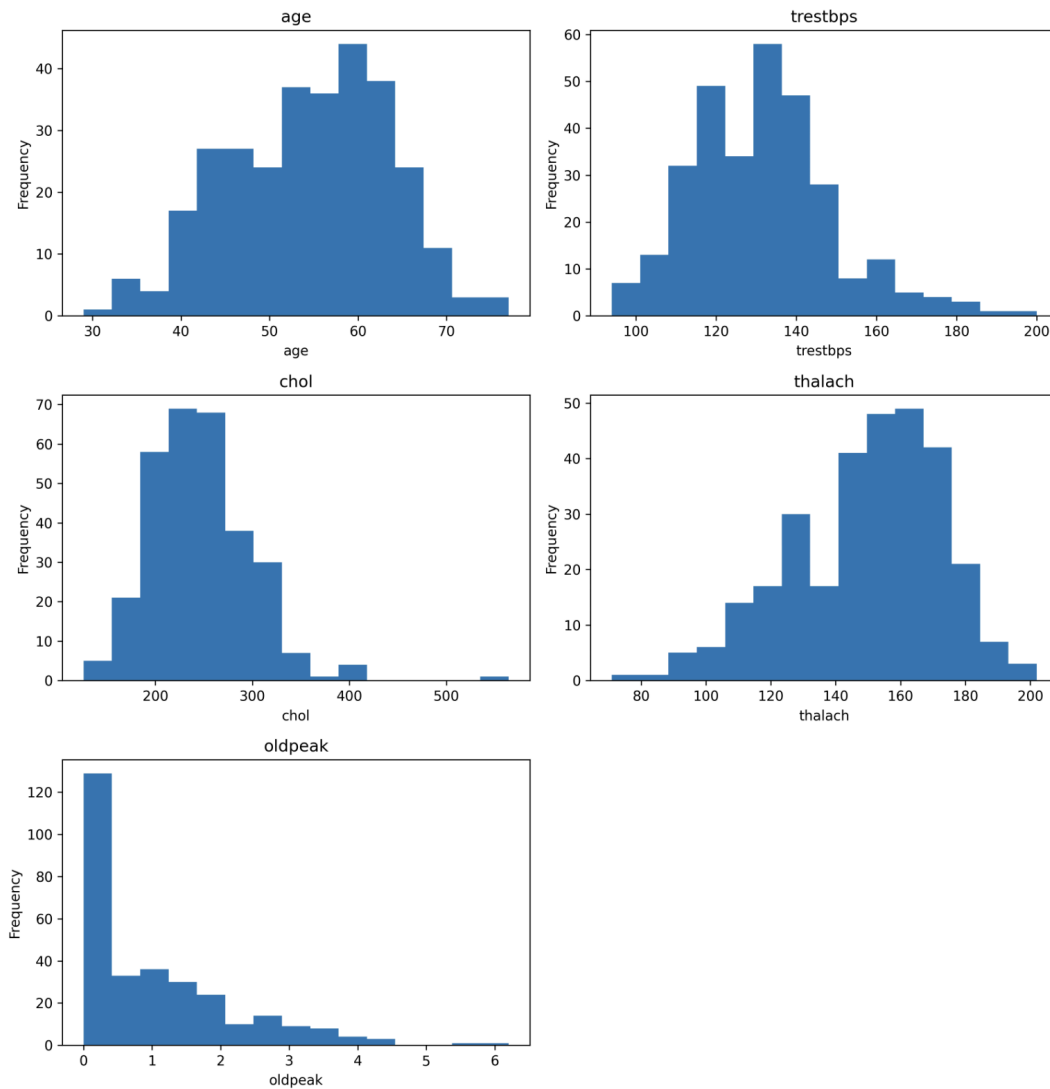
## Numerical Distributions

Histograms should be used to examine:

- age;
- resting blood pressure;
- cholesterol;
- maximum heart rate;
- ST depression.

The documented EDA identified a broad adult age range, substantial variation in maximum heart rate, large cholesterol observations, and right-skewness in oldpeak.

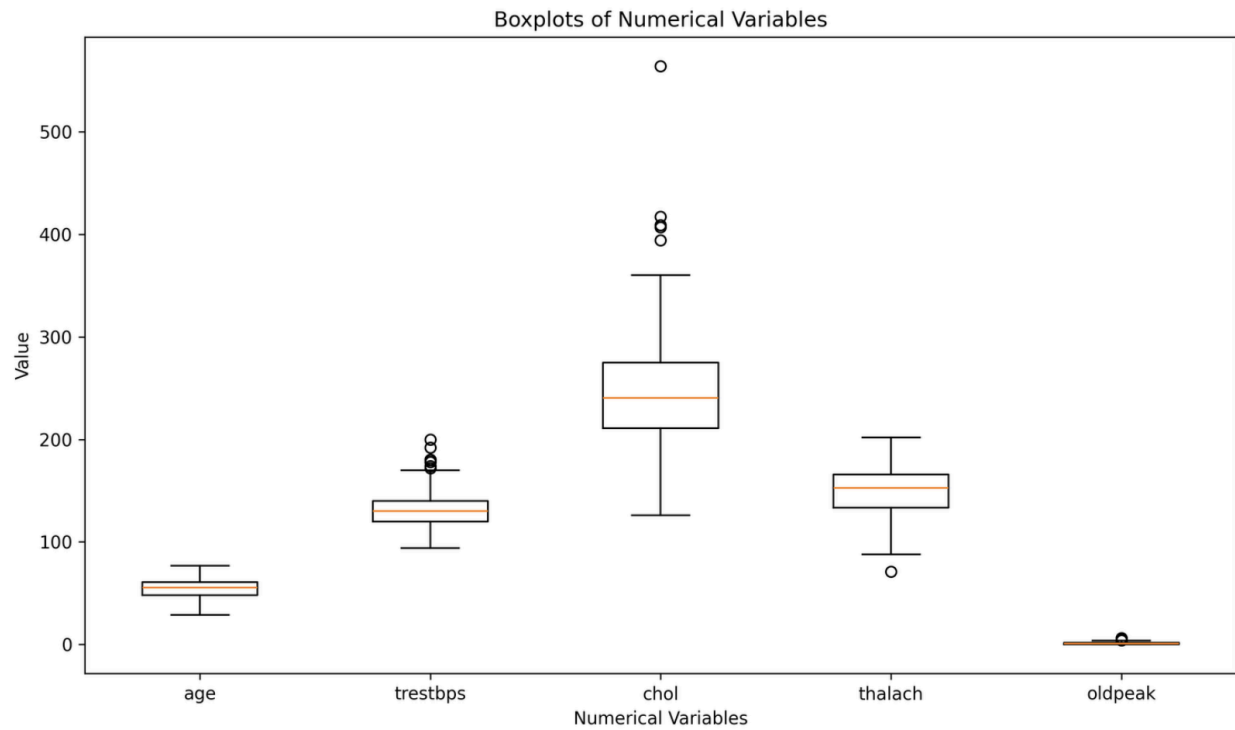
## Numerical Feature Distributions



## Boxplot Analysis

- Boxplots provide another view of distribution and potential extreme observations.

A clinically unusual measurement may represent a legitimate patient rather than an invalid observation.

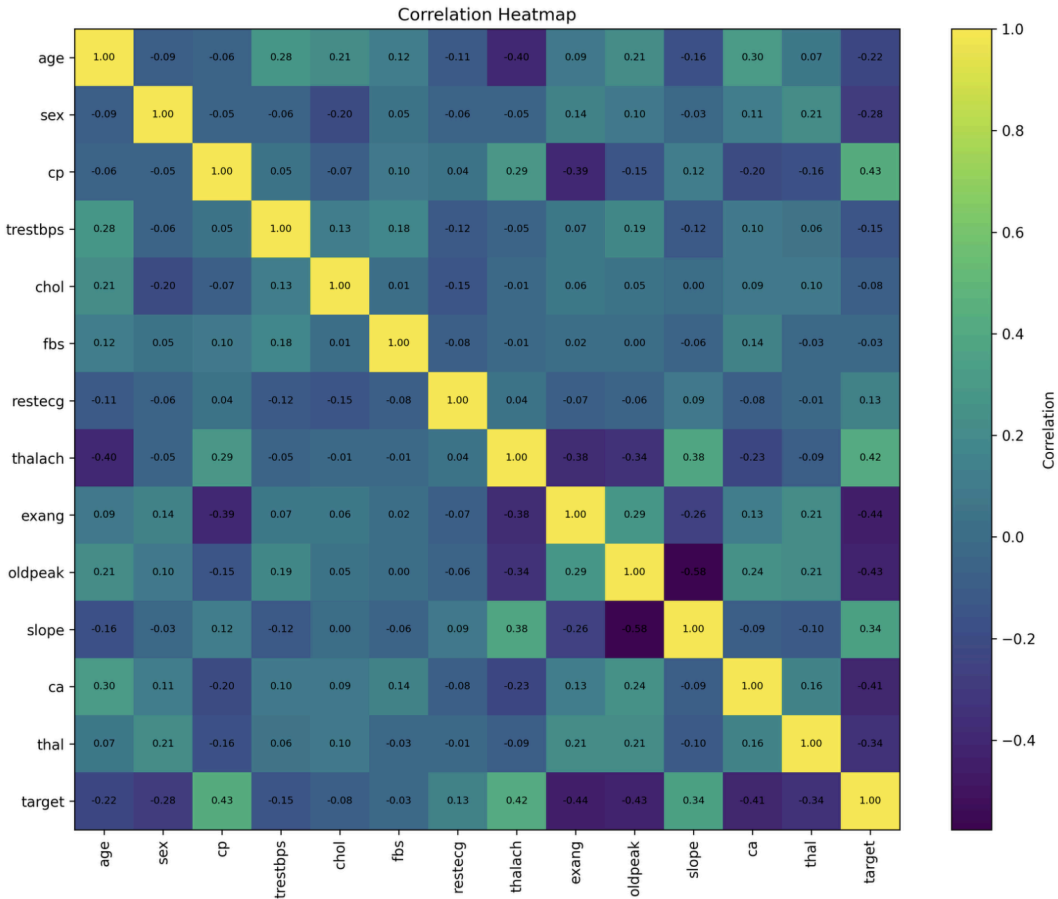


## Correlation Analysis

- Examine linear relationship among variables

The documented analysis indicates stronger relationships involving variables such as:

- chest-pain category (cp)
- exercise-induced angina (exang)
- ST depression (oldpeak)
- maximum heart rate (thalach)
- number of major vessels (ca)
- thal
- ST slope (slope)

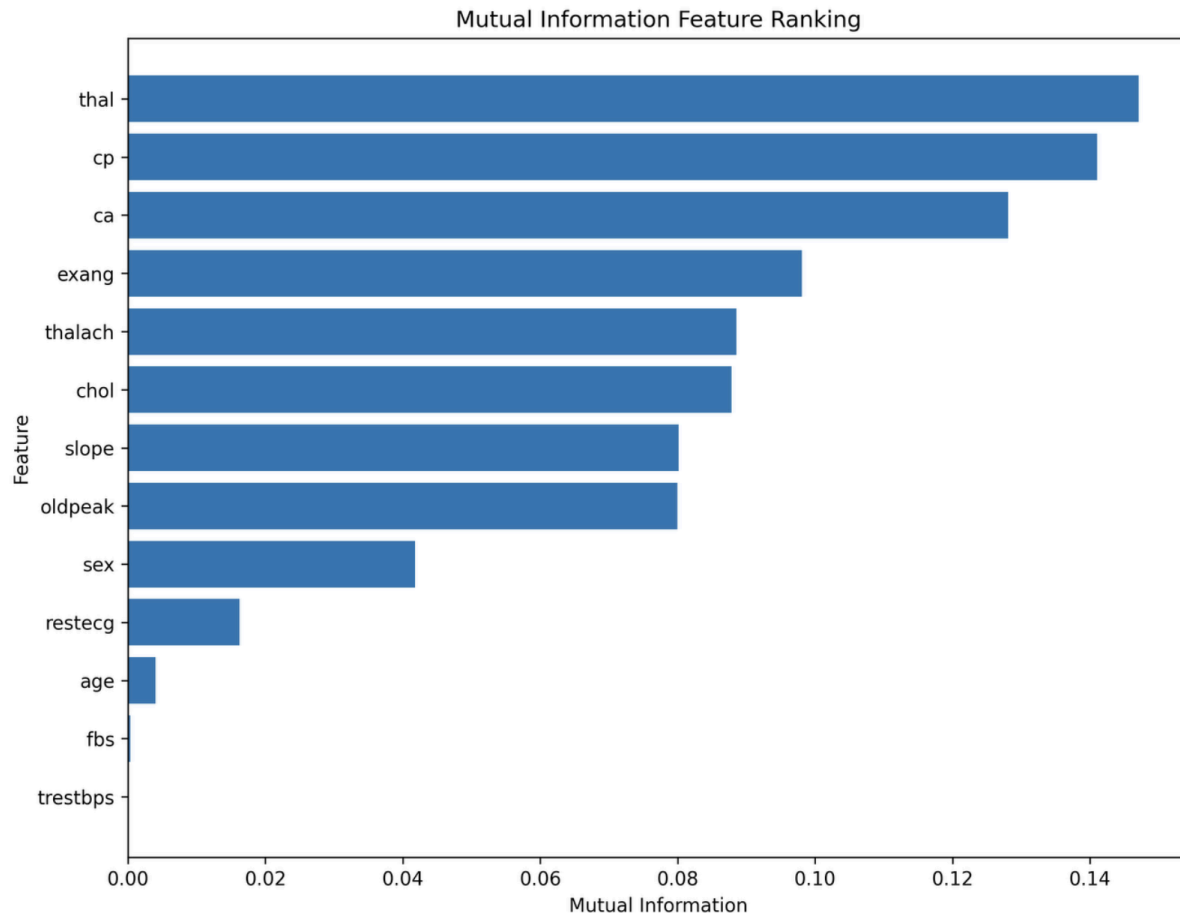


## Mutual Information

- Pearson correlation only notices straight-line relationships between a feature and the target. If the pattern curves or is more complicated, Pearson won't really see it. Mutual information can detect any kind of relationship, even nonlinear or irregular ones. It tells you how much a feature helps you learn about the target, no matter what the pattern looks like.

The analysis ranked the variables as follows:

1. thal
2. cp
3. ca
4. exang
5. thalach
6. chol
7. slope
8. oldpeak



## Data Understanding Findings

Important Conclusions:

1. The dataset is relatively small.
2. There are no missing values.
3. One duplicate must be removed.
4. The target is reasonably balanced.
5. Several clinical variables contain useful predictive information.
6. Some variables require categorical rather than continuous treatment.
7. Potential outliers require investigation rather than automatic removal.

These findings determine the decisions made in the next CRISP-DM phase.

## Phase 3: Data Preparation

### Duplicate Removal

```
df = df.drop_duplicates().reset_index(drop=True)
```



After removing the exact duplicate, the dataset contains 302 unique observations. Removing the duplicate makes sure the same patient isn't counted twice, which keeps the analysis and model results accurate.

## Encoding

The final preprocessing strategy separates:

### Numerical variables

- Age
- Trestbps
- Chol
- Thalach
- oldpeak

These variables are standardized.

### Categorical/binary variables

- sex
- Cp
- fbs
- restecg
- exang
- slope
- ca
- thal

Numerical features were scaled using StandardScaler. Categorical and yes/no features were turned into separate columns using OneHotEncoder, with **handle\_unknown="ignore"** so the pipeline won't break if new categories appear later. The preprocessing steps use StandardScaler, OneHotEncoder, ColumnTransformer, and Pipeline.

Putting all preprocessing inside the pipeline makes sure the model learns these transformations only from the training data during cross-validation, which helps prevent data leakage and keeps the evaluation fair.

The final pipeline uses:

- StandardScaler
- OneHotEncoder
- ColumnTransformer
- Pipeline

This ensures preprocessing is performed consistently and helps prevent data leakage during cross-validation.

## Feature Engineering

Transformations can be investigated, including:

- **Age squared**

```
df["age2"] = df["age"] ** 2
```

- **Blood Pressure / Age Ratio**

```
df["bp_age_ratio"] = df["trestbps"] / df["age"]
```

- **Cholesterol / Age Ratio**

```
df["chol_age_ratio"] = df["chol"] / df["age"]
```

- **Percentage of Predicted Maximum Heart Rate**

```
df["maxhr_pct_pred"] = (  
    df["thalach"] /  
    (220 - df["age"])  
)
```

- **Exercise Stress Indicator**

```
df["exercise_stress"] = (  
    df["exang"] * (df["oldpeak"] + 1)  
)
```

- **Binary Vessel Abnormality**

```
df["ca_any"] = (df["ca"] > 0).astype(int)
```

### **Feature Engineering Comparison Result:**

- Original Logistic Regression:
  - Accuracy: 83.8%
  - Recall: 86.2%
  - F1: 85.3%
  - ROC-AUC: 90.8%
- Engineering Logistic Regression:
  - Accuracy: 85.4%
  - Recall: 90.3%
  - F1: 87.1%
  - ROC-AUC: 90.9%

The engineered features were tested using the same validation process as the original features. Their performance was compared using accuracy, recall, F1-score, and ROC-AUC.

The goal was to see whether the extra feature transformations improved the model enough to justify the added complexity. When the engineered features didn't provide a meaningful improvement, the simpler feature set was kept.

### **Feature Selection**

Feature selection can be investigated through:

1. correlation
2. mutual information

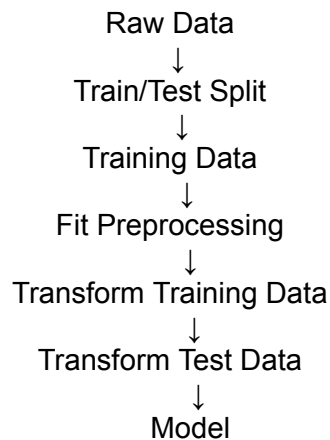
3. model coefficients
4. permutation importance
5. domain knowledge

The variables identified as useful include:

- number of major vessels (**ca**)
- chest pain (**cp**)
- thalassemia (**thal**)
- sex
- slope
- ST depression (**oldpeak**)

### Data Leakage Prevention

- A critical modeling principle is that transformations learned from the data must not use information from the test set.



This workflow prevents information from the evaluation data from influencing model training

## Phase 4 - Modeling

### Baseline

A basic baseline model was created before testing any real machine-learning algorithms. The baseline was a majority-class classifier, which always predicts the most common outcome in the dataset. Since it only repeats the most frequent label, it doesn't learn anything about how patient features relate to heart disease.

```
from sklearn.dummy import DummyClassifier

baseline = DummyClassifier(
    strategy="most_frequent"
)
baseline.fit(X_train, y_train)
Baseline_predictions = baseline.predict(X_test)
```

**Baseline achieved approx:**

- Accuracy: 54.1%
- ROC-AUC: 0.50

The baseline model serves as a reference point. Any useful machine-learning model should perform noticeably better than this. Since the dataset is fairly balanced, the majority-class classifier reaches only slightly above 50% accuracy. This makes it a weak baseline — but still a helpful one for comparison.

**Classification Models**

- Multiple classification algorithms were tested because it's not good to assume one model will be the best without actually trying them.

The models included:

1. Logistic Regression
2. K-Nearest Neighbors (KNN)
3. Support Vector Machine (SVM)
4. Decision Tree
5. Random Forest
6. Extra Trees
7. Gradient Boosting

The models were placed inside Scikit-learn preprocessing pipelines so that transformations were applied consistently.

**Logistic Regression**

- Logistic Regression fits well because the target is binary, the dataset is small, the model runs quickly, and its coefficients are easy to interpret. It also outputs probabilities, which are useful for ROC curves and adjusting decision thresholds.

**Tree-Based & Nonlinear Models**

- Decision Tree, Random Forest, Extra Trees, and Gradient Boosting were added to see whether nonlinear patterns or feature interactions could outperform the linear Logistic Regression model. The Decision Tree was limited to prevent overfitting on the small dataset.

**KNN & Support Vector Machine**

- SVM and KNN offer different ways of approaching the classification problem. Both require feature scaling because they rely on distances or boundary shapes that are affected by feature magnitude.

**Ensemble Modeling****Two ensemble approaches investigated**

1. **Soft Voting**
  - Logistic Regression
  - SVM
  - Random Forest

**Results:**

Accuracy  $\approx$  84.8%

F1  $\approx$  86.3%

ROC-AUC  $\approx$  91.1%

**2. Stacking**

- Stacking used Logistic Regression as the final estimator over the component models.

```
from sklearn.ensemble import StackingClassifier
stacking_model = StackingClassifier(
    estimators=[
        ("lr", logistic_model),
        ("svm", svm_model),
        ("rf", rf_model)
    ],
    final_estimator=LogisticRegression(
        max_iter=2000
    ),
    cv=5
)
```

**Results:**

Accuracy  $\approx$  85.1%

Recall  $\approx$  88.5%

F1  $\approx$  86.7%

ROC-AUC  $\approx$  91.1%

The ensemble models did not provide enough improvement over Logistic Regression to justify their additional complexity.

**Clustering**

- Classification uses the known target Clustering ignores the target and asks: "Are there naturally occurring patient subgroups?"

**K-means was applied after standardization**

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)

for k in range(2, 7):

    kmeans = KMeans(
        n_clusters=k,
        n_init=20,
        random_state=42
    )
```

```

labels = kmeans.fit_predict(X_scaled)

score = silhouette_score(
    X_scaled,
    labels
)

print(k, score)

```

Approximate silhouette results:

| K | Silhouette  |
|---|------------------------------------------------------------------------------------------------|
| 2 | <b>0.168</b>                                                                                   |
| 3 | 0.118                                                                                          |
| 4 | 0.123                                                                                          |
| 5 | 0.117                                                                                          |
| 6 | 0.116                                                                                          |

The two-cluster solution performs best according to silhouette score.

However, the silhouette value is relatively low.

## PCA Visualization

```

pca_2 = PCA(n_components=2)

X_2d = pca_2.fit_transform(X_scaled)

plt.figure(figsize=(9, 7))
sns.scatterplot(
    x=X_2d[:, 0],
    y=X_2d[:, 1],
    hue=y,
    palette="Set1"
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Projection of Heart Disease Dataset")
plt.show()

```

## Association Rule Mining

- Association mining can be used to discover combinations of patient characteristics associated with the target.

Example:

```
assoc = pd.DataFrame({
    "age_55plus": df["age"] >= 55,
    "male": df["sex"] == 1,
    "cp_nonanginal": df["cp"].isin([1, 2, 3]),
    "exercise_angina": df["exang"] == 1,
    "st_depression_gt1": df["oldpeak"] > 1,
    "ca_abnormal": df["ca"] > 0,
    "thal_abnormal": df["thal"].isin([1, 3]),
    "target_positive": df["target"] == 1
})
```

Association-rule metrics include:

**Support**

$$Support(A \rightarrow B) = P(A \cap B)$$

**Confidence**

$$Confidence(A \rightarrow B) = P(B|A)$$

**Lift**

$$Lift(A \rightarrow B) = \frac{P(B|A)}{P(B)}$$

### Strong Pattern

One of the strongest simple patterns found was:

- cp\_nonanginal  $\rightarrow$  target\_positive

Approximately:

- Support: 41.6%
- Confidence: 78.8%
- Lift: 1.45

This means that patients meeting the defined chest-pain condition had greater presence of the positive target than the dataset baseline

A two-feature pattern:

- male + cp\_nonanginal  $\rightarrow$  target\_positive

Had approximately:

- Support: 23.8%

- Confidence: 69.9%
- Lift: 1.28

## Similarity Search & LSH

The question becomes:

- Can we efficiently identify patients with similar profiles without comparing every patient to every other patient?

**Locality-Sensitive Hashing (LSH)** provides an educational introduction to scalable approximate similarity search.

The mixed medical data can first be transformed into a set of tokens representing:

- categorical values;
- age ranges;
- blood-pressure ranges;
- cholesterol ranges;
- heart-rate ranges;
- ST-depression ranges.

## LSH Concept

Traditional hashing tries to distribute similar records into different buckets but LSH does the opposite:

- Similar observations have a higher probability of being placed in the same bucket

This makes LSH useful for:

- Recommendation systems
- Duplicate detection
- Document similarity
- Patient-profile similarity
- Large-scale nearest-neighbor search

## Important Finding

- Dataset contains an exact duplicate: **Rows 163 and 164**
- LSH similarity experiment identifies these observations as having Jaccard similarity: **1.00**
- Other highly similar patient profiles had Jaccard similarities around: **0.86**

## Outlier Analysis

### Resting Blood Pressure

9 potential outliers including around:

- 172
- 178
- 180
- 192
- 200

### Maximum Heart Rate

- One potential low outlier: 71



## Oldpeak

Potential extreme values include:

- 4.2
- 4.4
- 5.6
- 6.2

## Phase 5 - Model Evaluation

Model evaluation was performed using two complementary strategies:

1. Stratified 10-fold cross-validation
2. An independent 80/20 holdout test set

Using both approaches provides a more reliable assessment than relying on a single train/test split.

### Cross-validation

- Stratified 10-fold cross-validation was used so each fold kept the same class proportions as the full dataset.

```
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(
    n_splits=10,
    shuffle=True,
    random_state=42
)
```

The primary metrics were:

- Accuracy
- Balanced Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

These metrics were used because accuracy alone isn't enough when false positives and false negatives have different consequences.

The analysis shows that Logistic Regression performed the best overall, with cross-validated results of about 85.7% accuracy, 89.7% recall, 87.4% F1 score, and 91.3% ROC-AUC.

### Holdout Test Evaluation

- After cross-validation, the final Logistic Regression pipeline was evaluated using a stratified 80/20 train-test split:

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X,  
y,  
test_size=0.20,  
stratify=y,  
random_state=42  
)
```

The holdout evaluation provides an estimate of how the trained model performs on observations that were not used during model fitting.

For the actual CSV run used to create your evaluation figures, the Logistic Regression model produced:

| Metric    | Holdout Result |
|-----------|----------------|
| Accuracy  | 85.25%         |
| Precision | 85.29%         |
| Recall    | 87.88%         |
| F1-score  | 86.57%         |
| ROC-AUC   | 89.72%         |

The cross-validation and holdout results differ slightly because they use different evaluation setups. Cross-validation averages performance across multiple stratified folds, while the holdout result comes from one separate test set.

## Confusion Matrix

```
from sklearn.metrics import confusion_matrix  
  
cm = confusion_matrix(  
    y_test,  
    y_pred  
)  
  
sns.heatmap(  
    cm,  
    annot=True,  
    fmt="d",
```

```
cmap="Blues"  
)  
  
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
plt.title("Logistic Regression Confusion Matrix")  
plt.show()
```

The actual holdout confusion matrix was:

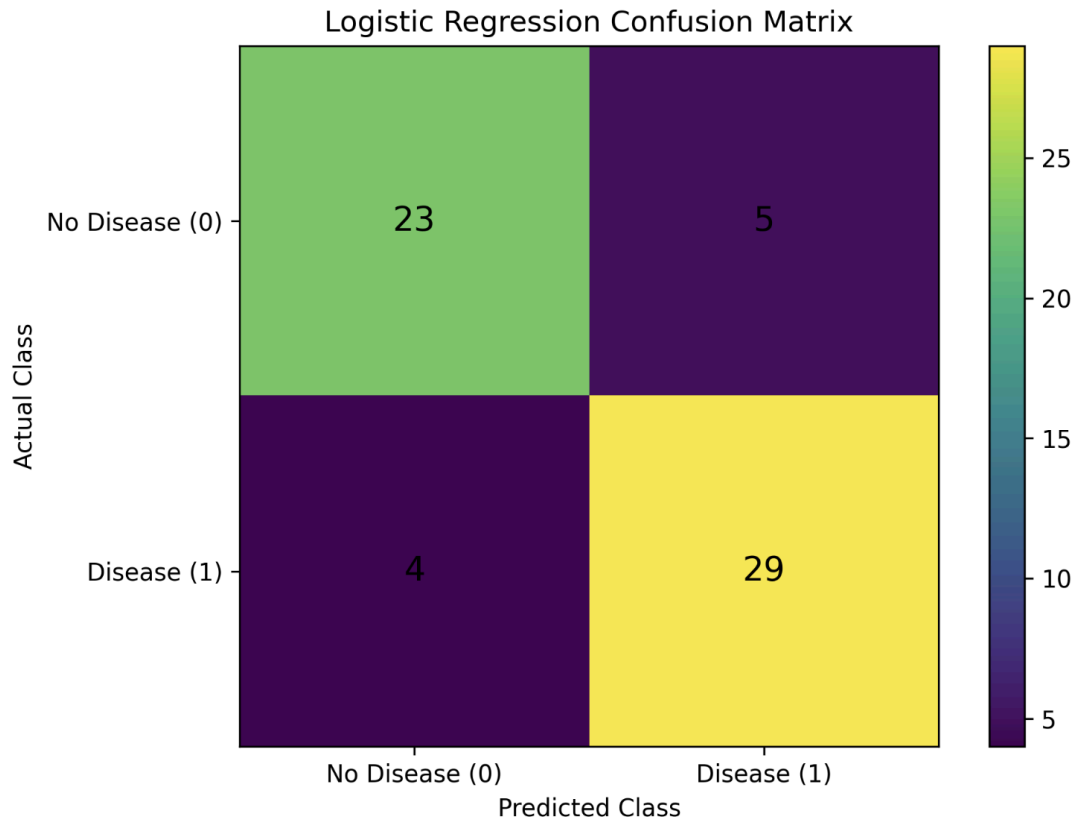
|                   | Predicted No Disease | Predicted Disease |
|-------------------|----------------------|-------------------|
| Actual No Disease | 23                   | 5                 |
| Actual Disease    | 4                    | 29                |

The corresponds to:

- 29 True Positives
- 23 True Negatives
- 5 False Positives
- 4 False Negatives

Interpretation

- The model correctly identified 29 positive heart-disease cases and 23 negative cases. It incorrectly classified 5 patients as positive when they were actually negative and missed 4 positive cases. The four false negatives are particularly important because the business objective places emphasis on identifying positive heart-disease cases.



## ROC Curve

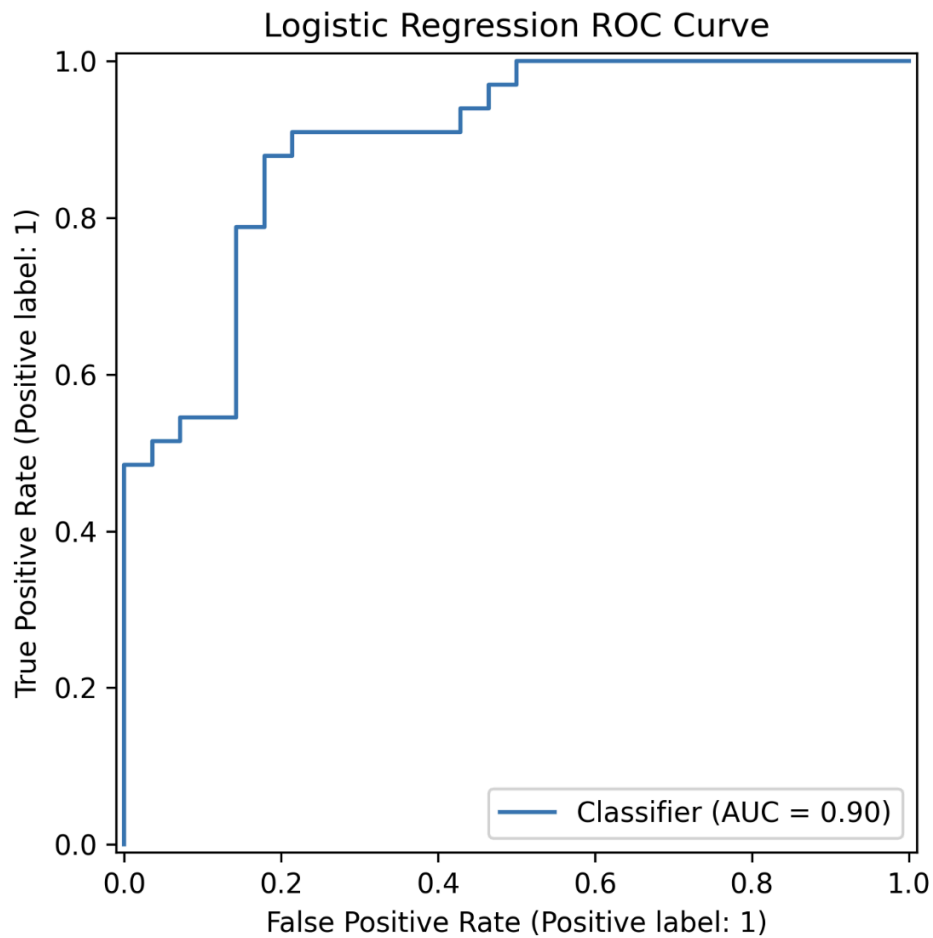
```
from sklearn.metrics import RocCurveDisplay
RocCurveDisplay.from_estimator(
    logistic_model,
    X_test,
    y_test
)

plt.title("ROC Curve -- Logistic Regression")
plt.show()
```

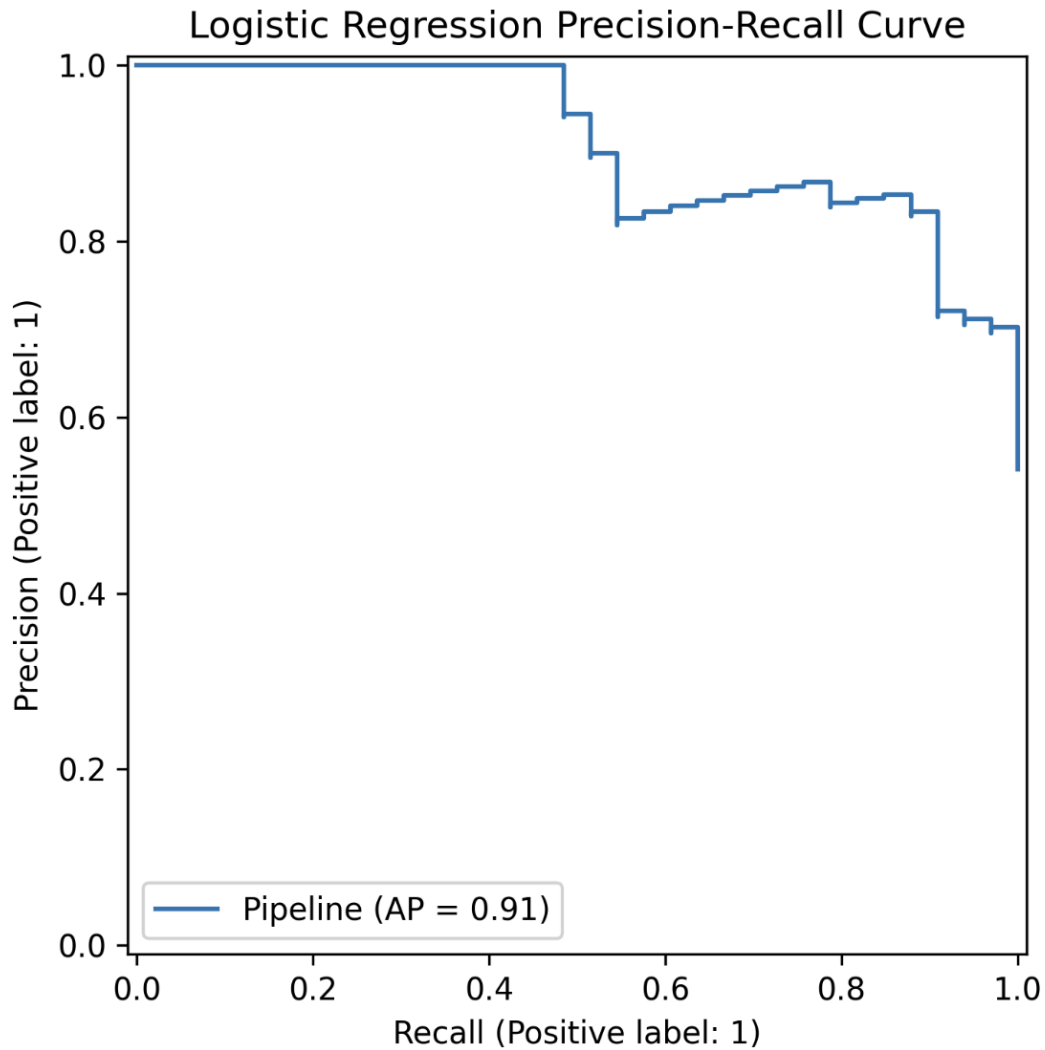
The ROC curve evaluates the model's ability to distinguish between the two target classes across different classification thresholds.

**ROC-AUC was approx: 0.897**

An ROC-AUC substantially above 0.50 indicates that the model reliably tells the two classes apart.



The ROC curve shows that the Logistic Regression model can tell the difference between patients who have the condition and those who don't. This is important for this heart-disease screening problem because false negatives are important.



The Precision-Recall Curve demonstrates how precision changes as recall increases. The default threshold of 0.50 isn't always the best choice. In a real screening situation, you would pick a threshold based on whether false negatives or false positives are more harmful.

### Model Comparison & Final Selection

Multiple models were tested using stratified 10-fold cross-validation, and their performance was compared using accuracy, recall, F1-score, and ROC-AUC.

## Model Comparison — Stratified 10-Fold Cross-Validation

| Model               | Accuracy | Recall | F1-Score | ROC-AUC |
|---------------------|----------|--------|----------|---------|
| Majority Baseline   | 54.4%    | 100.0% | 70.4%    | 50.0%   |
| Logistic Regression | 83.8%    | 86.2%  | 85.3%    | 90.8%   |
| KNN                 | 85.0%    | 87.0%  | 86.4%    | 89.6%   |
| SVM                 | 83.8%    | 85.4%  | 85.1%    | 90.8%   |
| Decision Tree       | 78.4%    | 77.0%  | 79.2%    | 80.8%   |
| Random Forest       | 83.4%    | 84.6%  | 84.5%    | 90.1%   |
| Extra Trees         | 83.4%    | 86.9%  | 85.1%    | 89.3%   |
| Gradient Boosting   | 79.7%    | 81.6%  | 80.9%    | 86.4%   |

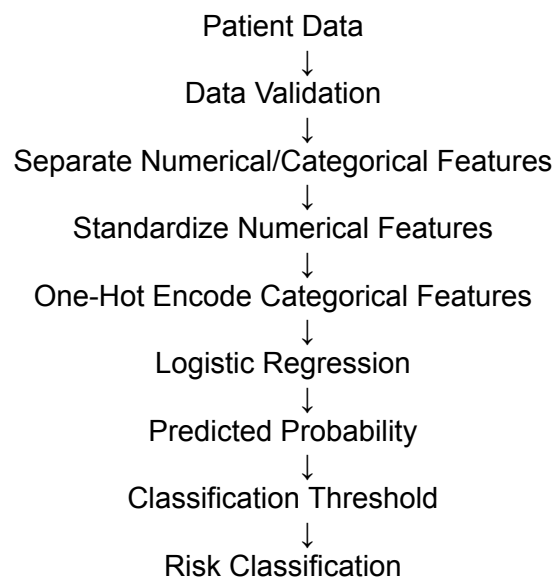
The results show that the more complex models didn't consistently perform better than the simpler ones. KNN had the highest accuracy and F1-score in cross-validation, while Logistic Regression and SVM scored the highest ROC-AUC. Logistic Regression performed the best overall because it offers a strong mix of good performance, easy interpretation, and fast computation.

## Phase 6 - Deployment

### Approach

- The selected Logistic Regression model can be deployed as a reproducible prediction pipeline.

The pipeline consists of:



A hypothetical application could accept patient characteristics and return:

- predicted class
- predicted probability
- classification threshold
- supporting model metrics

## Deployment Monitoring

Important areas to monitor include:

1. Data Quality
  - Watch for missing values, strange inputs, or changes in categories in new data.
2. Model performance
  - Regularly check recall, precision, F1-score, ROC-AUC, and calibration using new outcomes.
3. Data Drift
  - See whether new patient data starts to look different from the data the model was trained on.
4. Threshold Performance
  - Review the classification threshold based on how harmful false positives and false negatives are.
5. Fairness
  - Evaluate performance across different demographic and clinical groups to ensure the model works consistently.

## Clinical Deployment Limitations

The dataset contains only approximately 302 unique observations after duplicate removal, which limits generalizability. The model also lacks independent external validation.

Before real-world use, the workflow would require:





## **Final Recommendation:**

Logistic Regression is recommended as the main model for this project. It works well, is easy to understand, runs fast, and isn't too complex.

It performs much better than the baseline model, which shows that the patient features actually help predict the outcome. The ensemble models were good too, but they weren't better enough to justify their extra complexity.

So, Logistic Regression should be the primary model, with Random Forest and SVM kept as comparison models.

## **Conclusion:**

The CRISP-DM analysis shows that the Heart Disease dataset has real patterns that help predict the outcome. The EDA found several features that matter, such as chest-pain type, whether exercise causes discomfort, certain heart-test measurements, and overall heart rate.

Data preparation fixed the duplicate row and made sure both number-based and category-based features were handled correctly. After that, several models were tested using cross-validation and a separate test set.

Logistic Regression gave the best mix of performance and simplicity. On the holdout test set, it reached about 85% accuracy, 88% recall, 87% F1-score, and 90% ROC-AUC.

These results only apply to this dataset. The dataset is small, doesn't represent a wide population, and has not been tested on outside data. For these reasons, the model can't be used as a real medical diagnostic tool.

So, Logistic Regression should be used only as a learning or research model, not in real healthcare. Future work should include testing on new datasets, checking calibration, adjusting limits, checking fairness, running real-world tests, and getting clinical feedback.